

ADP4 Week 3: Big-O Notation, Searching & Sorting

Based on: Handwritten Class Notes & ADP470S Subject Guide (Term 1)

1. Big-O Notation & Time Complexity

Overview: Big-O notation describes the performance or complexity of an algorithm. It specifically describes the **worst-case scenario**, or the upper bound of the execution time as the input size (n) grows.

Complexity Classes (From your notes)

- $O(1)$ - **Constant Time:** The operation takes the same amount of time regardless of input size (e.g., accessing an array index).
- $O(\log n)$ - **Logarithmic Time:** The problem is cut in half at each step (e.g., Binary Search). This is very efficient.
- $O(n)$ - **Linear Time:** The time grows directly in proportion to the input. If you double the data, you double the time (e.g., Linear Search).
- $O(n^2)$ - **Quadratic Time:** Performance degrades quickly. Often seen in algorithms with nested loops (e.g., Bubble Sort, Selection Sort).
- $O(2^n)$ - **Exponential Time:** Your notes identify this as a "**bad algorithm**". The time doubles with each addition to the input. This was seen in the recursive Fibonacci sequence from Week 2.

Performance Graph Description

Your notes mention "Plot graphs." Visualizing complexity helps compare efficiency:

1. $O(1)$: A flat horizontal line.
 2. $O(\log n)$: A curve that rises slightly and then flattens out.
 3. $O(n)$: A straight diagonal line rising at a 45-degree angle.
 4. $O(n^2)$: A parabolic curve that shoots upward steeply (inefficient for large n).
-

2. Searching Algorithms

Linear Search

Definition: Iterates through a collection one element at a time until the target is found.

Complexity Analysis (From your notes):

- **Best Case:** $O(1)$ – The item is at the very first index.
- **Average Case:** $O(n/2)$ – On average, you search half the list. In Big-O, we drop constants, so this is $O(n)$.
- **Worst Case:** $O(n)$ – The item is at the very last index or not present at all.

Java Implementation:

Java

```
public int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i; // Return index
        }
    }
    return -1; // Not found
}
```

Binary Search

Definition: Used on **sorted/indexed** data structures. It repeatedly divides the search interval in half.

- **Complexity:** $O(\log n)$.
- **Note from handwriting:** "If it is indexed, it is searchable in log time." Trees (BST) are also searchable in log time.

3. Sorting Algorithms

Your notes list several sorting algorithms. Per the Subject Guide, **Bubble Sort** is a key focus for Term 1.

Comparison Table

Algorithm	Type	Time Complexity (Best)	Time Complexity (Worst)	Notes
Bubble Sort	Iterative	$O(n)$ (if modified)	$O(n^2)$	Swaps adjacent elements.

				Simple but slow.
Selection Sort	Iterative	$O(n^2)$	$O(n^2)$	Selects smallest item and moves to front.
Insertion Sort	Iterative	$O(n)$	$O(n^2)$	Good for small or nearly sorted lists.
Merge Sort	Recursive	$O(n \log n)$	$O(n \log n)$	"Fast." Divide and conquer. Uses extra memory.
Quick Sort	Recursive	$O(n \log n)$	$O(n^2)$	"Fast." Usually faster than Merge in practice, but unstable worst case.

Deep Dive: Selection Sort (From your notes)

Selection sort is an $O(n^2)$ algorithm. It works by repeatedly finding the minimum element (considering ascending order) from the unsorted part and putting it at the beginning.

- It is generally considered inefficient for large lists.

Deep Dive: Bubble Sort (Subject Guide Requirement)

The Subject Guide specifically lists "**Bubble Sort and Best-case, worst case and average case complexity**" as Subject Outcome 4 (SO4).

How it works:

1. Compare adjacent elements.
2. Swap them if they are in the wrong order.
3. Repeat passing through the list until no swaps are needed.

Java Implementation:

Java

```

void bubbleSort(int arr[]) {
    int n = arr.length;
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1]) {
                // swap arr[j+1] and arr[j]
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

```

4. Recursive vs. Iterative Recap (Fibonacci)

Your notes revisit the **Fibonacci Sequence**: 1, 1, 2, 3, 5, 8, 13.

- **Recursive Approach:** As noted in Week 2, this is $O(2^n)$ (Exponential). It recalculates the same values multiple times.
 - **Iterative Approach:** This is $O(n)$ (Linear). It stores the previous two numbers to calculate the next, avoiding redundant work.
 - **Lesson:** Just because you *can* use recursion doesn't mean you *should*. For Fibonacci, iteration is much better.
-

Next Steps for ADP4

According to the Subject Guide, upcoming topics for Term 1 include:

- **Arrays:** 2D and Multidimensional Arrays.
- **Linked Lists:** Introduction to Linear Linked Lists and limitations of Arrays.

Would you like me to generate a practice quiz based on these notes to help you prepare for the upcoming Term 1 test?